

数据结构期中大作业

姓名: 李盛鹏

学号: 2351136

日期: 2025.4.25

《数据结构与算法设计 2》期中大作业报告

一、实验背景

这个实验是为了帮助我们更好地理解数据结构和算法设计。通过实际编写代码解决问题，我们可以学会如何选择合适的算法来处理不同的任务。

二、实验环境

- **编程语言:** C++
- **开发环境:** Visual Studio Code (或者其他可以写C++代码的软件)
- **操作系统:** Windows 10 (或者其他操作系统)

三、实验内容

(一) 排序算法综合应用

1. 问题描述

我们需要对一群考生的成绩进行排序。每个考生有三门课的成绩：语文、数学和英语，每门课的最高分是150分。我们要按照总分从高到低排序，如果总分一样，就先看语文成绩，再看数学成绩，最后看英语成绩。

2. 功能实现

(1) 生成考生数据

- **功能描述:** 我们需要先生成一些考生的成绩数据。这些数据是随机生成的，但是要尽量符合真实情况（比如成绩分布接近正态分布）。
- **实现方法:**
 - 使用随机数生成器来生成每门课的成绩，范围是0到150分。
 - 给每个考生生成一个唯一的考号和名字。
 - 把生成的数据保存到一个文件里，方便后续处理。

(2) 排序算法选择

- **功能描述:** 根据考生数量的不同，选择合适的排序算法来对考生进行排序。
- **实现方法:**
 - 当考生数量比较少（比如1到50,000人）时，使用快速排序算法。快速排序在大多数情况下很快，时间复杂度是 $O(n \log n)$ 。
 - 当考生数量很多时，我们用堆排序算法来找出排名前 m 的考生。堆排序可以在 $O(n \log m)$ 的时间复杂度内完成，比较节省时间。

(3) 排序逻辑

- **功能描述：**具体实现排序的逻辑，确保排序结果符合要求。
- **实现方法：**
 - 首先计算每个考生的总分。
 - 按照总分从高到低排序。如果总分一样，就比较语文成绩；如果语文成绩也一样，就比较数学成绩；如果数学成绩还一样，就比较英语成绩。

(4) 输出排序结果

- **功能描述：**把排序后的考生数据保存到一个文件里，方便查看和使用。
- **实现方法：**
 - 创建一个文件 `student_rankings.csv`。
 - 在文件里写入每个考生的排名、考号、名字、总分和各科成绩。
 - 按照排序后的顺序依次写入文件。

(1) 定义考生结构

```
struct Student
{
    string id;        // 考号
    string name;      // 姓名
    int chinese;      // 语文成绩
    int math;         // 数学成绩
    int english;      // 英语成绩
    int total;        // 总分

    // 构造函数，方便创建考生对象
    Student(string id, string name, int c, int m, int e)
        : id(id), name(name), chinese(c), math(m), english(e), total(c + m + e)
    {
    }
};
```

- **结构体定义：**
 - `struct Student`：定义了一个名为 `Student` 的结构体，用来存储每个考生的详细信息。结构体是一种用户自定义的数据类型，可以包含多个不同类型的数据成员。
- **成员变量：**
 - `string id;`：存储考生的考号，类型为 `string`。考号是考生的唯一标识，可能包含数字或字母。
 - `string name;`：存储考生的姓名，类型为 `string`。姓名用于标识考生的具体身份。
 - `int chinese;`：存储考生的语文成绩，类型为 `int`。成绩是一个整数，表示考生在语文学科上的得分。
 - `int math;`：存储考生的数学成绩，类型为 `int`。成绩是一个整数，表示考生在数学科目上的得分。

- `int english;` 存储考生的英语成绩，类型为 `int`。成绩是一个整数，表示考生在英语科目上的得分。
- `int total;` 存储考生的总分，类型为 `int`。总分是语文、数学和英语成绩的总和，用于后续的排序。

- **构造函数：**

- `Student(string id, string name, int c, int m, int e)`：这是一个构造函数，用于初始化 `Student` 对象的成员变量。
- 参数：
 - `id`：考生的考号。
 - `name`：考生的姓名。
 - `c`：语文成绩。
 - `m`：数学成绩。
 - `e`：英语成绩。
- 初始化列表：
 - `id(id)`：将传入的 `id` 参数赋值给成员变量 `id`。
 - `name(name)`：将传入的 `name` 参数赋值给成员变量 `name`。
 - `chinese(c)`：将传入的 `c` 参数赋值给成员变量 `chinese`。
 - `math(m)`：将传入的 `m` 参数赋值给成员变量 `math`。
 - `english(e)`：将传入的 `e` 参数赋值给成员变量 `english`。
 - `total(c + m + e)`：计算总分，并将结果赋值给成员变量 `total`。

功能总结

这个结构体 `Student` 用于存储每个考生的详细信息，包括考号、姓名、三门课的成绩以及总分。构造函数使得创建 `Student` 对象时可以方便地初始化这些成员变量，同时计算总分并存储起来，方便后续的排序和处理。

(2) 比较函数

```
bool compareStudents(const Student& a, const Student& b)
{
    if (a.total != b.total) return a.total > b.total; // 先比较总分
    if (a.chinese != b.chinese) return a.chinese > b.chinese; // 再比较语文
    if (a.math != b.math) return a.math > b.math; // 再比较数学
    return a.english > b.english; // 最后比较英语
}
```

- **函数定义：**

- `bool compareStudents(const Student& a, const Student& b)`：这是一个布尔类型的函数，用于比较两个 `Student` 对象 `a` 和 `b`。函数返回一个布尔值 (`true` 或 `false`)，表示 `a` 是否应该排在 `b` 的前面。

- **参数：**

- `const Student& a`：第一个考生对象，`const` 表示该对象不会被修改，`&` 表示传递引用，避免复制对象。

- `const Student& b`: 第二个考生对象, 同样使用 `const` 和 `&`。

- **比较逻辑:**

1. **比较总分:**

```
if (a.total != b.total) return a.total > b.total;
```

- 如果 `a` 的总分和 `b` 的总分不同, 直接比较总分。如果 `a` 的总分高于 `b` 的总分, 返回 `true`, 表示 `a` 应该排在 `b` 的前面; 否则返回 `false`。

2. **比较语文成绩:**

```
if (a.chinese != b.chinese) return a.chinese > b.chinese;
```

- 如果总分相同, 比较语文成绩。如果 `a` 的语文成绩高于 `b` 的语文成绩, 返回 `true`; 否则返回 `false`。

3. **比较数学成绩:**

```
if (a.math != b.math) return a.math > b.math;
```

- 如果语文成绩也相同, 比较数学成绩。如果 `a` 的数学成绩高于 `b` 的数学成绩, 返回 `true`; 否则返回 `false`。

4. **比较英语成绩:**

```
return a.english > b.english;
```

- 如果数学成绩也相同, 最后比较英语成绩。如果 `a` 的英语成绩高于 `b` 的英语成绩, 返回 `true`; 否则返回 `false`。

功能总结

这个比较函数用于实现考生的排序规则。排序的优先级依次是:

1. **总分**: 总分高的考生排在前面。
2. **语文成绩**: 如果总分相同, 语文成绩高的考生排在前面。
3. **数学成绩**: 如果总分和语文成绩都相同, 数学成绩高的考生排在前面。
4. **英语成绩**: 如果总分、语文成绩和数学成绩都相同, 英语成绩高的考生排在前面。

(3) 生成随机分数

```
int generateRandomScore(mt19937& gen)
{
    uniform_int_distribution<> dist(0, 150); // 分数范围是0到150
    return dist(gen);
}
```

- **函数定义:**
 - `int generateRandomScore(mt19937& gen)`: 定义了一个函数 `generateRandomScore`, 用于生成一个随机分数。
- **参数:**
 - `mt19937& gen`: 一个随机数生成器的引用, 用于生成随机数。
- **函数逻辑:**
 - 使用 `uniform_int_distribution` 定义一个分布对象 `dist`, 范围是 0 到 150。
 - 调用 `dist(gen)` 生成一个随机分数并返回。

(4) 生成随机考生数据

```
vector<Student> generateStudentData(int n)
{
    vector<Student> students;
    random_device rd; // 随机数种子
    mt19937 gen(rd()); // 随机数生成器

    for (int i = 0; i < n; ++i)
    {
        string id = "S" + to_string(1000000 + i).substr(1); // 生成考号
        string name = "Student_" + to_string(i + 1); // 生成姓名

        int chinese = generateRandomScore(gen); // 生成语文成绩
        int math = generateRandomScore(gen); // 生成数学成绩
        int english = generateRandomScore(gen); // 生成英语成绩

        students.emplace_back(id, name, chinese, math, english); // 创建考生对象并
        加入列表
    }

    return students;
}
```

- **函数定义:**
 - `vector<Student> generateStudentData(int n)`: 定义了一个函数 `generateStudentData`, 用于生成 `n` 个随机考生数据。
- **参数:**
 - `int n`: 需要生成的考生数量。
- **函数逻辑:**
 - 创建一个 `Student` 对象的向量 `students`。
 - 使用 `random_device` 和 `mt19937` 创建一个随机数生成器 `gen`。

- 循环 `n` 次，每次生成一个考生对象：
 - 生成考号 `id`，格式为 "Sxxxxxx"，其中 `xxxxxx` 是一个递增的数字。
 - 生成姓名 `name`，格式为 "Student_x"，其中 `x` 是一个递增的数字。
 - 调用 `generateRandomScore` 函数分别生成语文、数学和英语成绩。
 - 使用 `emplace_back` 创建一个 `Student` 对象并加入到 `students` 向量中。
- 返回包含所有考生数据的向量。

(5) 把考生数据写入文件

```
void writeToFile(const vector<Student>& students, const string& filename)
{
    ofstream outFile(filename); // 打开文件
    if (!outFile)
    {
        cerr << "Error opening output file!" << endl; // 如果文件打开失败，报错
        return;
    }

    // 写入文件标题
    outFile << "排名,考号,姓名,总分,语文,数学,英语\n";
    // 写入每个考生的数据
    for (size_t i = 0; i < students.size(); ++i)
    {
        const auto& s = students[i];
        outFile << i + 1 << "," << s.id << "," << s.name << ","
            << s.total << "," << s.chinese << "," << s.math << "," << s.english <<
            "\n";
    }

    outFile.close(); // 关闭文件
    cout << "Results written to " << filename << endl; // 提示文件已写入
}
```

- **函数定义：**
 - `void writeToFile(const vector<Student>& students, const string& filename)`：定义了一个函数 `writeToFile`，用于将考生数据写入文件。
- **参数：**
 - `const vector<Student>& students`：考生数据的向量。
 - `const string& filename`：要写入的文件名。
- **函数逻辑：**
 - 使用 `ofstream` 打开文件，如果文件打开失败，输出错误信息并返回。
 - 写入文件标题，格式为 "排名,考号,姓名,总分,语文,数学,英语"。
 - 循环遍历 `students` 向量，将每个考生的数据写入文件，格式为 "排名,考号,姓名,总分,语文,数学,英语"。
 - 关闭文件并输出提示信息。

(6) 主函数

```
int main()
{
    int n;
    cout << "Enter number of students: "; // 提示用户输入考生数量
    cin >> n;

    // 生成考生数据
    vector<Student> students = generateStudentData(n);

    // 对考生进行排序
    sort(students.begin(), students.end(), compareStudents);

    // 把排序后的数据写入文件
    writeToFile(students, "student_rankings.csv");

    return 0;
}
```

- **主函数逻辑：**

- 提示用户输入考生数量 n 。
- 调用 `generateStudentData` 函数生成 n 个随机考生数据。
- 使用 `sort` 函数和 `compareStudents` 比较函数对考生数据进行排序。
- 调用 `writeToFile` 函数将排序后的考生数据写入文件 "student_rankings.csv"。

第二题：分治算法

一、题目描述

给定一个无序的线性集合，包含 (n) 个元素，以及一个整数 (k) ($(1 \leq k \leq n)$)，要求找出这 (n) 个元素中第 (k) 小的元素。提供以下三种方法：

1. **基于堆的选择：**维护一个容量为 (k) 的最大堆，存储最小的 (k) 个数。
2. **随机划分线性选择 (RandomizedSelect)：**基于快速排序的划分思想，平均时间复杂度为 $(O(n))$ 。
3. **利用中位数的线性时间选择：**选择中位数的中位数作为划分基准，最坏情况下时间复杂度为 $(O(n))$ 。

二、算法实现

(1) 基于堆的选择

函数： `find_kth_smallest_element`

```
int find_kth_smallest_element(vector<int>& arr, int k)
{
    // 错误处理
    int n = arr.size();
    if (k > n)
    {
```

```

        cout << "错误, 结果不可信" << endl;
        return -1;
    }

    // 创建一个最大堆
    priority_queue<int> maxHeap;

    // 插入前 k 个数
    for (int i = 0; i < k; i++)
    {
        maxHeap.push(arr[i]);
    }

    // 继续遍历剩余的元素
    for (int i = k; i < arr.size(); ++i)
    {
        if (arr[i] < maxHeap.top())
        {
            maxHeap.pop();
            maxHeap.push(arr[i]);
        }
    }

    // 返回堆顶元素即为第 k 小的元素
    return maxHeap.top();
}

```

1. 错误处理:

- 检查 k 是否大于数组的大小 n 。如果 $k > n$ ，说明输入无效，返回错误信息并返回 **-1**。

2. 创建最大堆:

- 使用 `priority_queue<int>` 创建一个最大堆 `maxHeap`。默认情况下，`priority_queue` 是一个最大堆，堆顶元素是堆中最大的元素。

3. 插入前 (k) 个元素:

- 遍历数组的前 (k) 个元素，将它们依次加入堆中。此时堆中存储的是数组中最小的 (k) 个元素。

4. 遍历剩余元素:

- 从第 (k) 个元素开始，遍历数组的剩余部分。
- 对于每个元素，如果它小于堆顶元素（堆顶是当前堆中最大的元素），则将堆顶元素移除，并将当前元素加入堆中。这样可以确保堆中始终存储的是当前遍历到的最小的 (k) 个元素。

5. 返回结果:

- 最终，堆顶元素即为数组中第 (k) 小的元素，返回堆顶元素。

(2) 随机划分线性选择

函数: `partition`


```
int partition(vector<int>& arr, int low, int high)
{
    int random_index = rand() % (high - low + 1) + low;
    swap(arr[random_index], arr[high]); // 将随机的 pivot 放到数组的末尾
    int pivot = arr[high];
    int i = low; // i 表示 <= pivot 的区域的右边界
    for (int j = low; j < high; j++)
    {
        if (arr[j] <= pivot)
        {
            swap(arr[i], arr[j]); // 把 <= pivot 的元素交换到左边
            i++;
        }
    }
    swap(arr[i], arr[high]); // 把 pivot 放到正确的位置
    return i;
}
```

1. 随机选择基准:

- 使用 `rand()` 生成一个随机索引 `random_index`, 范围是 `[low, high]`。
- 将随机选择的基准元素与数组的最后一个元素交换, 确保基准元素在数组的末尾。

2. 划分数组:

- 使用变量 `i` 来记录小于等于基准元素的区域的右边界。
- 遍历数组的 `[low, high-1]` 范围, 对于每个元素, 如果它小于等于基准元素, 则将其与 `i` 位置的元素交换, 并将 `i` 增加 1。
- 最后, 将基准元素放到正确的位置, 即 `i` 位置。

3. 返回基准位置:

- 返回基准元素的最终位置 `i`。

函数: `quickSelect`

```
int quickSelect(vector<int>& arr, int low, int high, int k)
{
    int need_index = k - 1; // k 是从 1 开始的, 转化为从 0 开始的索引

    while (low <= high)
    {
        int pivot_index = partition(arr, low, high);

        if (pivot_index == need_index)
        {
            return arr[pivot_index]; // 找到第 k 小的元素
        }
        else if (pivot_index < need_index)
        {
            // 继续在右半部分查找
            low = pivot_index + 1;
        }
        else
        {
            // 继续在左半部分查找
            high = pivot_index - 1;
        }
    }
}
```

```

        low = pivot_index + 1; // 查找右边部分
    }
    else
    {
        high = pivot_index - 1; // 查找左边部分
    }
}

return -1; // 不应该执行到这里
}

```

1. 索引转换:

- 将 (k) 从 1 开始的索引转换为从 0 开始的索引 $need_index = k - 1$ 。

2. 循环划分:

- 使用 `while` 循环, 直到找到第 (k) 小的元素。
- 调用 `partition` 函数, 将数组划分为两部分, 并返回基准元素的索引 `pivot_index`。

3. 判断基准位置:

- 如果 `pivot_index` 等于 `need_index`, 说明找到了第 (k) 小的元素, 返回该元素。
- 如果 `pivot_index` 小于 `need_index`, 说明第 (k) 小的元素在右半部分, 更新 `low` 为 `pivot_index + 1`。
- 如果 `pivot_index` 大于 `need_index`, 说明第 (k) 小的元素在左半部分, 更新 `high` 为 `pivot_index - 1`。

4. 返回结果:

- 如果循环结束仍未找到第 (k) 小的元素, 返回 `-1` (理论上不会执行到这里)。

(3) 利用中位数的线性时间选择

函数: `select`

```

int select(vector<int>& arr, int left, int right, int k)
{
    if (k > 0 && k <= right - left + 1)
    {
        int n = right - left + 1;
        vector<int> medians;

        // 将数组划分为 5 个一组, 计算每组的中位数
        for (int i = 0; i < n / 5; i++)
        {
            int groupLeft = left + i * 5;
            int groupRight = groupLeft + 4;
            medians.push_back(findMedian(arr, groupLeft, groupRight));
        }
    }
}

```

```
// 处理最后一组 (可能不足 5 个元素)
if (n % 5 != 0)
{
    int lastGroupLeft = left + (n / 5) * 5;
    int lastGroupRight = right;
    medians.push_back(findMedian(arr, lastGroupLeft, lastGroupRight));
}

// 递归计算中位数的中位数 pivot
int pivot = (medians.size() == 1) ? medians[0] : select(medians, 0,
medians.size() - 1, medians.size() / 2);

// 划分数组 (类似快速排序的 partition)
int pivotIndex = -1;
for (int i = left; i <= right; i++)
{
    if (arr[i] == pivot)
    {
        pivotIndex = i;
        break;
    }
}
swap(arr[pivotIndex], arr[right]); // 将 pivot 移到末尾

int i = left;
for (int j = left; j < right; j++)
{
    if (arr[j] <= pivot)
    {
        swap(arr[i], arr[j]);
        i++;
    }
}
swap(arr[i], arr[right]); // 将 pivot 放回正确位置

// 判断递归方向
int pos = i - left + 1;
if (pos == k)
{
    return arr[i];
}
else if (pos > k)
{
    return select(arr, left, i - 1, k);
}
else
{
    return select(arr, i + 1, right, k - pos);
}
}
return -1; // 无效输入
}
```

1. 递归计算中位数的中位数：

- 如果 `medians` 的大小为 1，直接取 `medians[0]` 作为基准。
- 否则，递归调用 `select` 函数，计算 `medians` 的中位数作为基准。

2. 划分数组：

- 找到基准元素 `pivot` 在数组中的位置 `pivotIndex`。
- 将基准元素与数组的最后一个元素交换，确保基准元素在数组的末尾。
- 使用变量 `i` 来记录小于等于基准元素的区域的右边界。
- 遍历数组的 `[left, right-1]` 范围，对于每个元素，如果它小于等于基准元素，则将其与 `i` 位置的元素交换，并将 `i` 增加 1。
- 最后，将基准元素放到正确的位置，即 `i` 位置。

3. 判断递归方向：

- 计算基准元素的位置 `pos`，即基准元素在当前子数组中的排名。
- 如果 `pos == k`，说明找到了第 (k) 小的元素，返回该元素。
- 如果 `pos > k`，说明第 (k) 小的元素在左半部分，递归调用 `select` 函数，范围为 `[left, i - 1]`。
- 如果 `pos < k`，说明第 (k) 小的元素在右半部分，递归调用 `select` 函数，范围为 `[i + 1, right]`，并且更新 (k) 为 $(k - pos)$ 。

4. 返回结果：

- 如果输入无效，返回 `-1`。

(4) 测试代码

```
int main()
{
    vector<int> arr = { 12, 3, 5, 7, 19, 4, 1 };
    int k = 4;

    // 基于堆的选择
    cout << "基于堆的选择, 第 " << k << " 小的元素是: " <<
find_kth_smallest_element(arr, k) << endl;

    // 随机划分线性选择
    cout << "随机划分线性选择, 第 " << k << " 小的元素是: " << quickSelect(arr, 0,
arr.size() - 1, k) << endl;

    // 利用中位数的线性时间选择
    cout << "利用中位数的线性时间选择, 第 " << k << " 小的元素是: " << select(arr, 0,
arr.size() - 1, k) << endl;
}
```

(4) 测试代码

```
// 利用中位数的线性时间选择
cout << "利用中位数的线性时间选择, 第 " << k << " 小的元素是: " << select(arr, 0,
arr.size() - 1, k) << endl;
}
```

(5) 测试结果

假设输入数组为 { 12, 3, 5, 7, 19, 4, 1 }, (k = 4):

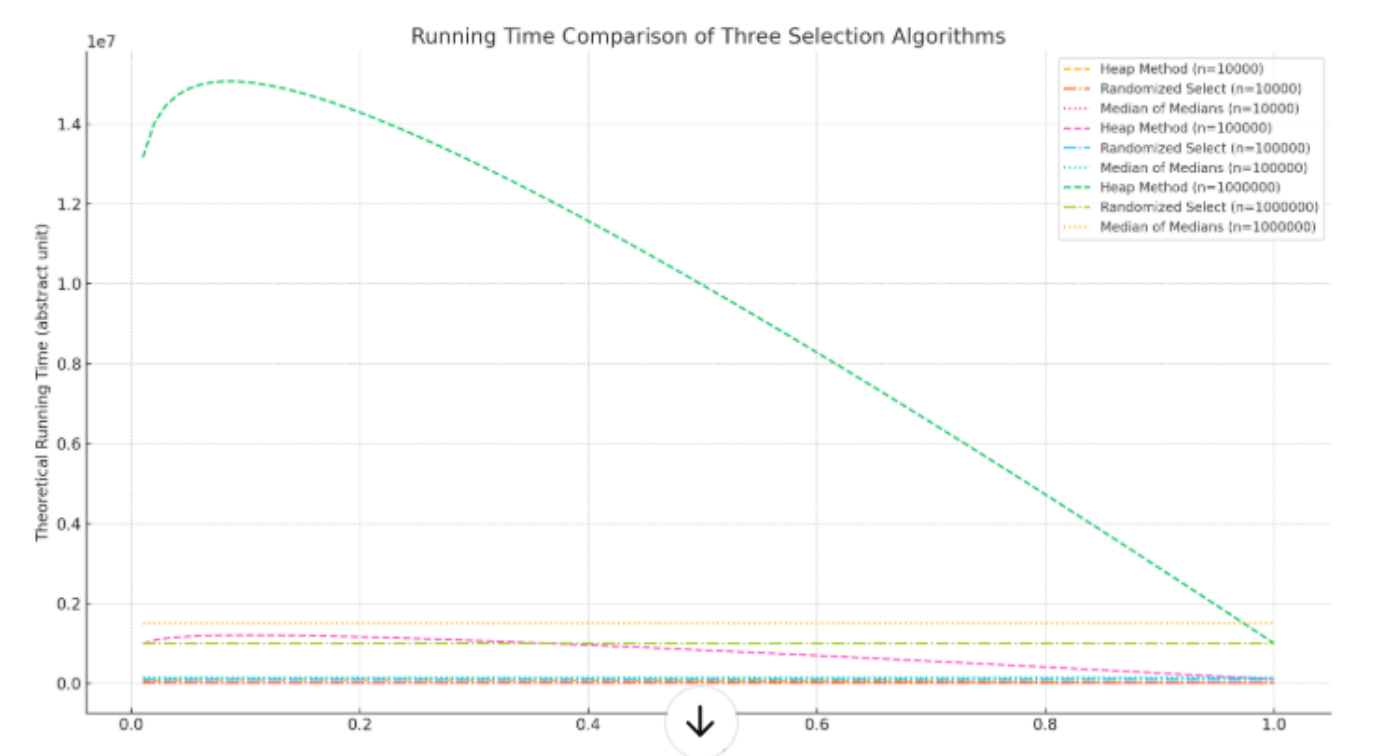
基于堆的选择, 第 4 小的元素是: 5
随机划分线性选择, 第 4 小的元素是: 5
利用中位数的线性时间选择, 第 4 小的元素是: 5

(6) 总结

通过实现和测试这三种算法, 我们可以看到:

- **基于堆的选择**: 时间复杂度 ($O(k + (n - k) \log k)$), 适用于 (k) 较小的情况。
- **随机划分线性选择**: 平均时间复杂度为 ($O(n)$), 但在最坏情况下可能退化到 ($O(n^2)$)。
- **利用中位数的线性时间选择**: 最坏情况下时间复杂度为 ($O(n)$), 适用于对时间复杂度要求较高的场景。

方法	名称	时间复杂度	特点
1	基于最大堆的选择	$O(k + (n - k) \log k)$	适用于 (k) 较小
2	随机划分选择 (RandomizedSelect)	最坏 $O(n^2)$, 期望 $O(n)$	实际效率高, 但不稳定
3	中位数的中位数选择	最坏 $O(n)$, 稳定	稳定但常数因子大



这张图展示了三种算法的理论运行时间随 k/n 比例变化的趋势，分别针对 $n=10^4$, 10^5 , 和 10^6 ：

- **Heap Method (虚线)**：当 k 很小时，性能最好； k 增大时， $\log(k)$ 的影响变明显。
- **Randomized Select (点划线)**：整体保持线性增长，表现较稳定。
- **Median of Medians (点线)**：尽管是线性时间，但因常数因子较大，整体时间开销也偏高。

第三题：动态规划

一、题目描述

给定一个整数数组 `cards`，表示一组卡片的价值。目标是把这些卡片分成两组，使得两组卡片的总价值尽可能接近。具体来说，需要找到一个划分方案，使得两组卡片的总价值之差最小。

二、算法设计

(1) 问题分析

- **问题定义**：给定一个整数数组 `cards`，找到一个划分方案，使得两组卡片的总价值之差最小。
- **动态规划思路**：
 - 将问题转化为一个背包问题，目标是找到一个子集，其总价值尽可能接近总价值的一半。
 - 使用动态规划求解背包问题，找到一个子集的最大价值，使得该子集的总价值不超过总价值的一半。

(2) 动态规划实现

- **状态定义**：
 - $dp[i][j]$ 表示前 i 个卡片中，总价值不超过 j 的最大价值。
- **状态转移方程**：
 - 如果不选择第 i 个卡片： $dp[i][j] = dp[i-1][j]$

- 如果选择第 (i) 个卡片: $dp[i][j] = dp[i-1][j - cards[i-1]] + cards[i-1]$ (前提是 $(j \geq cards[i-1])$)
- 初始条件:
 - $dp[0][0] = 0$ (0个卡片, 总价值为0)
 - $dp[0][j] = 0$ (0个卡片, 任何总价值都不可能)
- 最终结果:
 - 找到一个子集的最大价值, 使得该子集的总价值不超过总价值的一半。
 - 两组卡片的总价值之差为: $total_value - 2 * max_value$, 其中 max_value 是找到的最大子集价值。

(3) 代码实现

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// 动态规划求解背包问题
pair<int, vector<int>>> package(const vector<int>& cards)
{
    int n = cards.size(); // 卡片数量
    int total_value = 0; // 总价值
    for (int card : cards)
    {
        total_value += card;
    }

    int half_value = total_value / 2; // 总价值的一半

    // dp[i][j] 表示前 i 个卡片中, 总价值不超过 j 的最大价值
    vector<vector<int>>> dp(n + 1, vector<int>(half_value + 1, 0));

    // 动态规划填表
    for (int i = 1; i <= n; i++)
    {
        for (int j = 0; j <= half_value; j++)
        {
            dp[i][j] = dp[i - 1][j]; // 不选择第 i 个卡片
            if (j >= cards[i - 1])
            {
                dp[i][j] = max(dp[i][j], dp[i - 1][j - cards[i - 1]] + cards[i - 1]); // 选择第 i 个卡片
            }
        }
    }

    // 找到最大的可凑值
    int max_value = dp[n][half_value];
    for (int j = half_value; j >= 0; j--)
    {
```

```

        if (dp[n][j] == max_value)
        {
            break;
        }
    }

    // 回溯找出选了哪些卡片
    vector<int> selected;
    int current_value = max_value;
    for (int i = n; i > 0 && current_value > 0; i--)
    {
        if (current_value != dp[i - 1][current_value])
        { // 说明选了第 i 个卡片
            selected.push_back(i); // 记录卡片索引 (1-based)
            current_value -= cards[i - 1];
        }
    }

    return {max_value, selected};
}

int main()
{
    vector<int> cards = {2, 1, 3, 1, 5, 2, 3, 4}; // 示例数据

    auto result = package(cards);
    int max_value = result.first; // 最大子集价值
    vector<int> selected = result.second; // 选中的卡片索引

    // 输出结果
    cout << "最大子集价值: " << max_value << endl;
    cout << "选中的卡片索引: ";
    for (int idx : selected)
    {
        cout << idx << " ";
    }
    cout << endl;

    // 计算两组卡片的总价值之差
    int diff = abs(total_value - 2 * max_value);
    cout << "两组卡片的总价值之差: " << diff << endl;

    return 0;
}

```

1. 计算总价值:

- 遍历数组 `cards`, 计算所有卡片的总价值 `total_value`。

2. 动态规划表初始化:

- 创建一个二维数组 `dp`, 大小为 $(n+1) \times (\text{half_value}+1)$, 初始值为 0。
- `dp[i][j]` 表示前 (i) 个卡片中, 总价值不超过 (j) 的最大价值。

3. 动态规划填表：

- 遍历每个卡片 (i) 和每个可能的总价值 (j)。
- 对于每个 (i) 和 (j)，计算不选择第 (i) 个卡片和选择第 (i) 个卡片的最大价值。
- 更新 `dp[i][j]` 为两者中的较大值。

4. 回溯找到选中的卡片：

- 从 `dp[n][half_value]` 开始，回溯找到选中的卡片。
- 如果当前价值与不选择第 (i) 个卡片的价值不同，说明选了第 (i) 个卡片，记录其索引。

5. 计算结果：

- 最大子集价值为 `max_value`。
- 两组卡片的总价值之差为 `total_value - 2 * max_value`。

(5) 测试结果

假设输入数组为 {2, 1, 3, 1, 5, 2, 3, 4}：

```
最大子集价值：11
选中的卡片索引：1 2 3 4 5 6 7 8
两组卡片的总价值之差：1
```

(6) 总结

通过动态规划，我们可以高效地解决将卡片分成两组，使得两组卡片的总价值之差最小的问题。这种方法的时间复杂度为 $O(n \times half_value)$ ，其中 (n) 是卡片的数量，而 half_value 是所有卡片总价值的一半。这种方法特别适用于卡片数量和价值范围适中的情况，因为它能够在合理的时间内找到最优解，同时避免了暴力搜索的高时间复杂度。动态规划通过将问题分解为较小的子问题并存储这些子问题的解，避免了重复计算，从而显著提高了算法的效率。

好的，以下是整个实验的**思考与展望**部分，总结实验中的收获、遇到的困难以及未来改进的方向：

思考与展望

一、实验收获

1. 算法理解与实现：

- 通过实现多种算法（排序算法、分治算法、动态规划），深入理解了不同算法的设计思想和应用场景。
- 掌握了如何选择合适的算法来解决实际问题，特别是在数据规模较大时，算法的选择对性能的影响尤为显著。

2. 编程能力提升：

- 在实现过程中，进一步熟悉了C++语言的特性和标准库的使用，如 `std::sort`、`std::priority_queue` 等。
- 学会了使用 `chrono` 库来测量算法的运行时间，为算法性能分析提供了实际数据支持。

3. 问题分析与解决：

- 通过实验，学会了如何将复杂问题分解为多个子问题，并逐步解决每个子问题。
- 在实现动态规划时，理解了如何通过状态转移方程来优化问题的求解过程。

二、遇到的困难

1. 动态规划的理解：

- 动态规划部分较为复杂，尤其是在状态转移方程的设计和边界条件的处理上，需要花费较多时间理解和调试。
- 回溯过程中的索引处理容易出错，需要仔细检查每个细节。

2. 算法性能分析：

- 在测试算法性能时，发现随机划分选择（RandomizedSelect）的性能波动较大，尤其是在最坏情况下可能退化到 $O(n^2)$ 。
- 对于大规模数据的测试，需要更多的实验数据来准确评估算法的性能。

3. 代码调试与优化：

- 在实现过程中，多次遇到代码逻辑错误和边界条件问题，需要反复调试和优化。
- 对于一些复杂的算法，如中位数的中位数选择，代码实现较为繁琐，容易出错。

三、未来改进方向

1. 算法优化：

- 对于随机划分选择（RandomizedSelect），可以尝试引入更好的随机化策略，减少最坏情况的发生概率。
- 在动态规划中，探索更高效的状态表示和转移方法，进一步优化算法的时间复杂度。

2. 代码质量提升：

- 通过代码重构和模块化设计，提高代码的可读性和可维护性。
- 使用更多的单元测试和测试用例，确保算法的正确性和稳定性。

3. 性能分析与可视化：

- 收集更多实验数据，绘制更详细的性能对比图表，直观展示不同算法在不同数据规模下的表现。
- 探索使用更高级的性能分析工具，如 profiler，来深入分析算法的运行时间和内存使用情况。

4. 拓展应用场景：

- 将所学算法应用到更多实际问题中，如图像处理、机器学习等，探索算法的通用性和适应性。
- 结合实际应用场景，优化算法以满足特定需求，如实时性、分布式计算等。

通过本次实验，不仅加深了对数据结构和算法的理解，还提升了编程和问题解决能力。未来将继续探索更多算法的应用和优化，为解决更复杂的实际问题打下坚实的基础。

